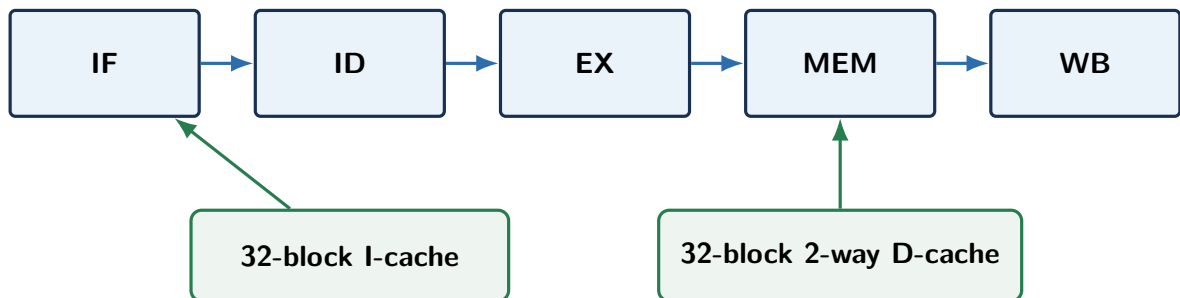


Hardware Implementation of a Pipelined RISC-V Processor

Digital System Design Final Project Report



Group 1
Members B12901008, B12901137
Course Digital System Design
Date June 14, 2026

Abstract

This project implements a synthesizable pipelined RV32 processor that operates correctly with fast, fixed-latency, and random-latency external memories. The processor contains a five-stage core, complete forwarding and stall control, separate blocking instruction and data caches, and a write-back flush mechanism. Three extension topics were explored: compressed instructions, multiplication, and branch prediction. The final implementation supports the required RV32I subset, sixteen RV32C compressed instructions, and a multicycle MUL. We also move unconditional JAL resolution from EX to ID and use a dedicated JAL immediate path to reduce its control penalty without extending the main EX path.

The submitted gate-level design runs at a 2.7 ns testbench clock period with a synthesized cell area of 769950.83 μm^2 . Its measured gate-level times are 193798.947 ns, 34423.347 ns, and 722164.647 ns for QSort, Conv, and LFSR_HIST, respectively, yielding the official area–time product of 3.7094×10^{21} .

2.7 ns
Gate clock

769,951
Cell area (μm^2)

RV32I+C+M
ISA support

3.7094e21
Final score

Contents

1	Specification and Design Goals	1
2	Processor Architecture	1
3	Hazards and Control Flow	2
4	Cache Design and Optimization	3
5	ISA Extensions	4
6	Synthesis and Timing Closure	5
7	Verification and Final Results	6
8	Engineering Process and Lessons Learned	7
9	Conclusion and Future Work	7

1 Specification and Design Goals

1.1 Required behavior

The project specification requires a pipelined RISC-V processor with at least five stages, synchronous active-low reset, and the prescribed RV32I subset: integer arithmetic and logic, shifts, comparison, conditional branches, JAL/JALR, LW/SW, NOP, and the project-specific FLUSH instruction. The design must resolve structural, data, and control hazards using hardware forwarding and stall logic rather than software-inserted NOPs.

Instruction and data memories are external to CHIP. Correctness is required under `fast_memory`, `slow_memory`, and `slow_memory_random_latency`. The latter has long variable latency, so cache behavior and request stability dominate performance and correctness. When the final FLUSH instruction retires, all dirty cache lines must have been written back before `o_done` is asserted.

1.2 Evaluation targets

The timing target is a post-synthesis testbench clock period below 3.5 ns, no negative setup or hold slack, and no gate-level timing violation. Final performance is evaluated by

$$\text{Score} = A \times T_{\text{QSort}} \times T_{\text{Conv}} \times T_{\text{LFSR_HIST}},$$

where lower area and lower simulation time are preferred. The extension evaluation additionally checks branch prediction, compressed instructions, and multiplication at gate level.

1.3 Final report requirement

The project description explicitly requires the June 14 submission folder to contain the final presentation slides and a detailed report of 8–16 pages. It does not prescribe a chapter template. This report therefore emphasizes architecture, implementation decisions, experimental evidence, synthesis results, and lessons learned.

2 Processor Architecture

2.1 Top-level organization

The top-level CHIP contains one pipelined core, a read-only I-cache, and a write-back D-cache. Both caches communicate with independent 128-bit external memory ports. The final RTL parameters and instantiated cache modules are shown in Table 1.

Table 1: Final implementation configuration.

Component	Final configuration	Purpose
Core	Five-stage in-order RV32	Simple, verifiable pipeline
I-cache	32 blocks, direct-mapped	Remove repeated instruction misses
D-cache	32 blocks, 2-way, write-back	Reduce data conflict misses
Cache line	128 bits (four words)	Match external memory interface
MUL	Two-cycle multicycle path	Meet timing without a long EX path
Control prediction	Always not taken	Lowest hardware cost
JAL resolution	ID stage	One-cycle JAL penalty

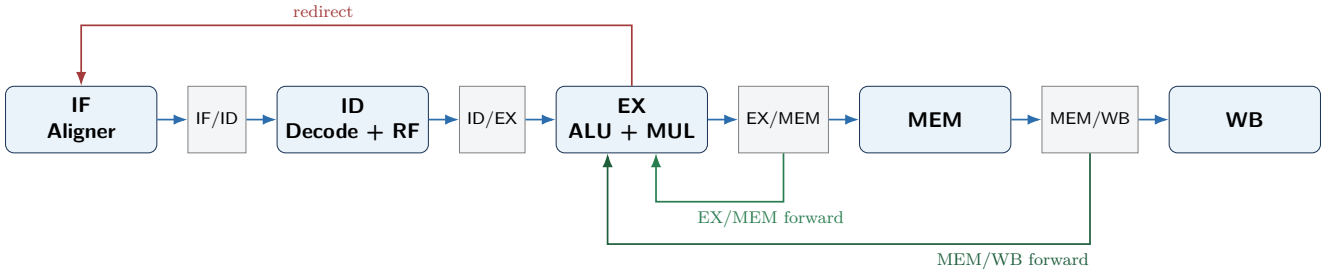


Figure 1: Core pipeline and major feedback paths.

2.2 Pipeline operation

The IF stage fetches and aligns either a 16-bit or 32-bit instruction. ID decodes the instruction, reads the register file, generates immediates, and resolves JAL. EX performs ALU operations, branch comparison, JALR target generation, forwarding selection, and multicycle multiplication. MEM issues load/store requests through the D-cache, while WB commits the selected result to the register file.

Each stage carries a `valid` bit and only valid instructions may cause architectural side effects. This makes bubbles and flushes explicit and avoids depending on arbitrary payload values after reset. The pipeline is globally held while an outstanding memory transaction is incomplete. During a MUL stall, earlier instructions are allowed to drain while younger stages are held.

3 Hazards and Control Flow

3.1 Data and structural hazards

EX operands pass through two forwarding multiplexers. An ALU result can be forwarded from EX/MEM; a completed ALU or load result can be forwarded from MEM/WB. The same forwarded second operand is used as store data, preventing a common store-after-produce failure. Loads require one bubble when the following instruction consumes the destination because load data is not available at the start of the next EX cycle.

The separate I-cache and D-cache eliminate contention between instruction fetch and data access. A cache miss still creates a structural dependency on its external memory port. The core therefore keeps requests and addresses stable and stalls until the corresponding `mem_ready` is observed.

Table 2: Hazard handling summary.

Hazard	Detection or mechanism	Action
EX/MEM RAW	Destination matches EX source; result is not a load	Forward
MEM/WB RAW	Destination matches EX source	Forward
Store-data RAW	Destination matches store's second source	Forward
Load-use	ID/EX load destination matches IF/ID source	Insert bubble
Memory wait	Active memory request without ready	Hold pipeline
MUL wait	Valid MUL has not completed	Hold younger stages
Taken branch/JALR	EX redirect is valid	Flush younger instructions
JAL	Decode detects valid JAL	Redirect in ID, flush IF only

3.2 Control hazards and ID-stage JAL

The baseline control policy is always not taken. Conditional branches and JALR are resolved in EX because they require forwarded register operands. Incorrectly fetched younger instructions are invalidated when EX redirects the PC.

JAL does not require register operands. We therefore move its target calculation to ID, reducing its penalty from two flushed instructions to one. A dedicated immediate concatenation computes the J-type offset directly from instruction bits, bypassing the general immediate multiplexer:

$$\text{jal_target} = \text{ifid_pc} + \{\{\text{inst}[31]\}, \text{inst}[19:12], \text{inst}[20], \text{inst}[30:21], 0\}.$$

Branches and JALR remain in EX because moving their forwarding and comparison logic to ID would threaten the clock period.

4 Cache Design and Optimization

4.1 Cache microarchitecture

The I-cache is a direct-mapped read-only blocking cache. The D-cache is a two-way set-associative blocking cache with valid and dirty bits. On a dirty miss, the old 128-bit line is written back before the requested line is refilled. Word selection occurs inside the cached line. The D-cache replacement policy selects a way within the indexed set, and the FLUSH FSM walks through every set and way, writing back all dirty lines.

The core’s done flag and the D-cache flush-done flag are combined at the top level. Consequently, `o_done` cannot become high before architectural execution is complete and memory is coherent with the golden checker.

4.2 Cache sweep methodology

Before committing to RTL configurations, we built a C++ instruction-set and cache simulator that reads the supplied patterns. It counts instruction/data accesses, misses, writebacks, and expected slow-memory delay for several block counts and associativities. Promising cases were then tested in RTL because the analytical model cannot capture synthesized area, critical paths, or pipeline-control overhead.

Table 3: Selected cache miss-rate observations from the software sweep.

Pattern	Cache	16 blocks, 1-way	32 blocks, 1-way	64 blocks, 2-way
QSort	I miss rate	2.5498%	0.0541%	0.0541%
QSort	D miss rate	6.7210%	3.8209%	0.9843%
Conv	I miss rate	3.6880%	0.9164%	0.8941%
Conv	D miss rate	21.2606%	2.9163%	1.3641%
LFSR_HIST	I miss rate	18.4577%	0.0130%	0.0130%
LFSR_HIST	D miss rate	37.5730%	36.5347%	34.7826%

The largest gain is obtained by increasing the I-cache from 16 to 32 blocks: LFSR_HIST’s instruction miss rate collapses from 18.46% to 0.013%. A larger two-way D-cache substantially helps QSort and Conv, while LFSR_HIST remains dominated by a data access pattern with poor locality. Balancing area against miss-rate improvement, the final submitted design uses 32 direct-mapped I-cache blocks and 32 two-way D-cache blocks. Higher associativity was rejected because extra tag comparisons, data multiplexers, and replacement state provided little additional benefit for the supplied workloads.

Table 4: Selected RTL cache experiment cycles (QSort / Conv / LFSR_HIST).

Configuration	QSort	Conv	LFSR_HIST
I16 / D16, both 1-way	112,201	21,300	1,226,355
I32 / D16, both 1-way	87,807	20,873	278,220
I32 / D32, both 1-way	79,718	14,678	277,771
I32 / D64, both 1-way	73,561	10,965	277,357

5 ISA Extensions

5.1 Compressed instruction support

The IF stage determines instruction length from the low two bits of the current halfword. A compressed instruction is expanded into an equivalent 32-bit RV32I instruction before decode, allowing the remainder of the pipeline to be shared. Sixteen compressed operations are supported: `c.nop`, `c.add`, `c.mv`, `c.addi`, `c.andi`, `c.slli`, `c.srli`, `c.srai`, `c.sw`, `c.lw`, `c.beqz`, `c.bnez`, `c.j`, `c.jal`, `c.jr`, and `c.jalr`.

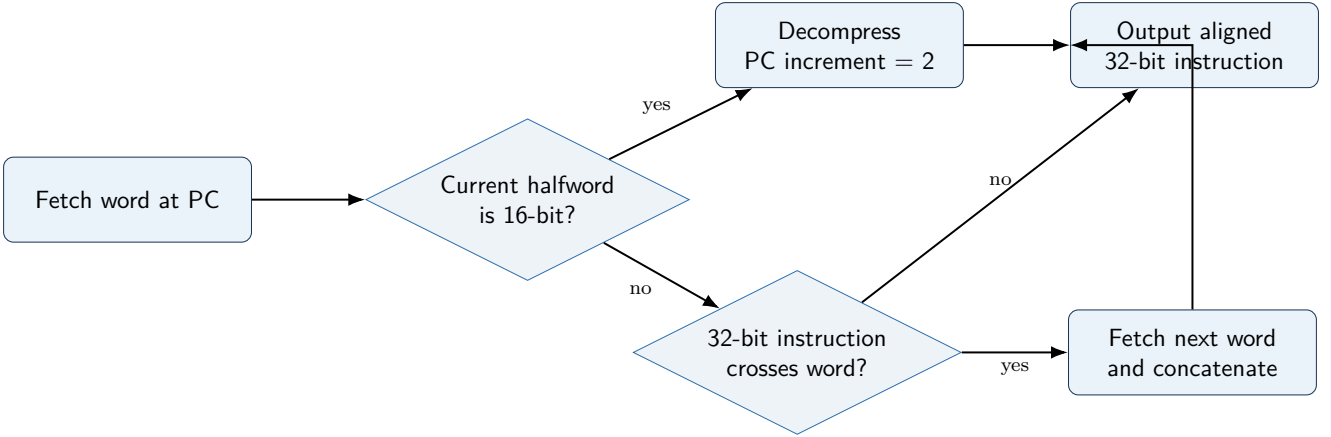


Figure 2: Instruction alignment and decompression flow.

If a 32-bit instruction begins in the upper halfword, the aligner saves that halfword and fetches the next word before concatenation. This also handles a cross-cache-line instruction. The pipeline carries an explicit PC increment (2 or 4) so that compressed JAL/JALR write the correct link address. The aligned 32-bit fast path remains single-cycle for normal final patterns.

5.2 Multicycle multiplication

A single-cycle combinational multiplier would lengthen EX and make the target period difficult to achieve. The implemented unit captures already-forwarded operands into dedicated registers, computes the product, and captures the result after a two-cycle multicycle interval. IF, ID, and the active EX instruction are held while older EX/MEM and MEM/WB instructions drain.

The synthesis constraints match this microarchitecture:

```

set mul_cycles 2
set_multicycle_path $mul_cycles -setup \
  -from [get_cells -hier *mul_a_reg*] \
  -to [get_cells -hier *mul_result_reg*]
set_multicycle_path 1 -hold \
  -from [get_cells -hier *mul_a_reg*] \
  -to [get_cells -hier *mul_result_reg*]
  
```

The official multiplication gate-level pattern completes in 738.147 ns at a 2.7 ns clock period.

5.3 Branch-prediction exploration

We evaluated always-not-taken, always-taken, backward-taken/forward-not-taken (BTFNT), global counters, bimodal tables, and gshare using branch traces from the supplied workloads. The specialized BrPred test favors a small bimodal table, but that improvement does not generalize to the three performance workloads. In particular, LFSR_HIST branches depend strongly on generated data, so more state does not imply better prediction.

Table 5: Software branch-prediction accuracy.

Method	BrPred	QSort	Conv	LFSR_HIST
Always not taken	37.97%	72.44%	63.58%	85.46%
Always taken	62.03%	27.56%	36.42%	14.54%
BTFNT	73.42%	72.44%	66.05%	85.46%
Global 3-bit counter	68.35%	70.48%	63.58%	85.44%
8-entry 2-bit bimodal	93.67%	74.08%	63.58%	83.60%

The final core retains always-not-taken because it has zero predictor-table area, is already strong on QSort and LFSR_HIST, and avoids adding another redirect path to IF. We instead applied the lower-risk ID-stage JAL optimization. The final BrPred gate-level result is 558 cycles; the hasHazard reference pattern takes 2,375 cycles.

6 Synthesis and Timing Closure

6.1 Flow and constraints

The design is synthesized in Synopsys Design Compiler with the TSMC 0.13 μm cell library. The flow analyzes and elaborates the single RTL source, applies SDC constraints, runs `compile_ultra`, flattens the design, performs timing-focused incremental optimization, and emits Verilog, SDF, DDC, SDC, and reports.

The submitted run uses a 2.7 ns clock, 0.1 ns clock uncertainty, and 0.5 ns clock latency. External memory inputs and outputs receive half-cycle maximum delay constraints because the testbench memory interface operates on the opposite clock phase. Reset release and multiplication are explicitly constrained as multicycle paths.

6.2 Synthesis results

The reported critical path begins at external `mem_ready_D` and ends at a D-cache data-array enable flip-flop. It meets timing exactly after including the 1.35 ns external input delay. This confirms that the cache control/refill interface, rather than the integer ALU, is the dominant timing concern.

For comparison, a synthesized one-way D-cache candidate at 2.71 ns had a similar cell area ($767791.73 \mu\text{m}^2$) but a deeper 20-level critical path through the D-cache write path. The two-way implementation’s modest area increase was accepted for its miss-rate benefit and shorter reported critical logic depth.

Report caveat. The final QoR has no setup or hold violations, but the constraint report contains one small maximum-capacitance violation (-0.03 in the library’s capacitance units) on a register-file net. The automatically reported `max_area` and `max_leakage_power` entries are artifacts of setting their

Table 6: Final 2-way D-cache synthesis summary.

Metric	Result
Clock period	2.70 ns
Critical path slack / total negative slack	0.00 ns / 0.00 ns
Setup / hold violating paths	0 / 0
Logic levels on reported CLK critical path	8
Combinational / sequential cell count	28,294 / 11,595
Total cell area	769950.832599 μm^2
Estimated total dynamic power	99.6629 mW
Estimated leakage power	147.5071 μW

optimization target to zero, rather than functional or timing failures. The power values are vectorless estimates, so they are useful for comparison but not workload-accurate power.

7 Verification and Final Results

7.1 Verification strategy

Verification proceeded from isolated behavior to combined regression:

1. Validate the required RV32I subset with noHazard and targeted ALU tests.
2. Exercise EX/MEM forwarding, MEM/WB forwarding, store-data forwarding, load-use bubbles, branch forwarding, JAL/JALR redirects, and flushes.
3. Run all tests against fast, fixed-latency, and random-latency memories.
4. Validate compressed lower/upper halfwords, cross-word instructions, cross-line instructions, and $PC + 2$ compressed links.
5. Validate MUL with forwarded operands and surrounding memory traffic.
6. Re-run RTL and post-synthesis gate-level simulations for final patterns.

Small directed patterns were essential because a full hasHazard failure often appears far after the original bad redirect or forwarded value. Examples include adjacent branch dependencies, load-to-branch, store-before-flush, JALR with negative immediate, and D-cache conflict/flush cases.

7.2 Official final results

Table 7: Submitted gate-level results at a 2.7 ns testbench clock.

Category	Pattern or metric	Result
Branch/control	BrPred	558 cycles
Branch/control	hasHazard	2,375 cycles
Compressed	Compression simulation time	1874.847 ns
Multiplication	MUL simulation time	738.147 ns
Performance	QSort simulation time	193798.947 ns
Performance	Conv simulation time	34423.347 ns
Performance	LFSR_HIST simulation time	722164.647 ns
Performance	Cell area	769950.832599 μm^2
Performance	Area-time product	3.7094×10^{21}

Relative to the checkpoint result, the final design increases cell area from approximately $400226.55 \mu\text{m}^2$

to $769950.83\mu\text{m}^2$. This increase is primarily the cost of larger caches, two-way D-cache structures, the compressed decoder/aligner, and the multiplier. In return, the final performance workloads complete much faster under random-latency memory, and all three extension categories are supported in one synthesized core.

8 Engineering Process and Lessons Learned

8.1 Experiment-driven decisions

The most effective optimization process was to separate three questions: functional correctness, workload benefit, and timing feasibility. C++ models quickly rejected unpromising cache and predictor configurations without requiring synthesis. RTL cycle counters then exposed pipeline stalls that a miss-rate-only model could not represent. Finally, gate reports identified cache control paths and external delay assumptions that determined the actual clock period.

Several plausible ideas were intentionally not included in the final design. A branch target buffer for JAL could remove recurring jump penalties but adds a lookup and target path in IF. A D-cache write buffer could overlap dirty writeback with refill but needs extra storage and more complex request ordering. Large dynamic branch predictors performed well on the BrPred trace but offered inconsistent benefit on the performance suite. These choices reflect the project score’s simultaneous sensitivity to area, runtime, and timing closure.

8.2 Use of AI-assisted tools

AI-assisted coding was used as an engineering aid for generating initial RTL structure, writing targeted patterns, automating shell/Makefile workflows, and building C++ cache and branch-predictor sweep tools. Generated changes were introduced incrementally and checked after each step. Functional simulation, gate-level simulation, and synthesis reports remained the source of truth; this was especially important for pipeline control and timing constraints, where syntactically valid code can still be architecturally incorrect.

8.3 Work assignment

Table 8: Team work assignment.

Member	Primary contributions
B12901008	Cache-size experiments, branch-prediction experiments, compressed instruction extension, multiplication extension
B12901137	Cache-associativity experiments, assembly-code analysis, ID-stage JAL resolution, synthesis and timing optimization

9 Conclusion and Future Work

We completed a synthesizable five-stage RISC-V processor with robust hazard handling, independent caches, RV32C support, and multicycle multiplication. The final gate-level design meets the 2.7 ns clock target with zero negative setup/hold slack and achieves an official performance score of 3.7094×10^{21} . The largest performance improvement came from matching cache capacity and associativity to the supplied workload behavior; the lowest-risk control improvement was resolving JAL one stage earlier.

Future work should first remove the remaining maximum-capacitance violation and add activity-annotated power analysis. Architectural extensions should be evaluated in this order: a small

nonblocking or write-buffered D-cache, a carefully timed IF-stage JAL target buffer, and selective dynamic prediction only if its runtime benefit outweighs area and redirect-path cost. The current experiment infrastructure provides a practical basis for making those decisions quantitatively.

References

1. Digital System Design Final Project, *Pipelined RISC-V Design*, project description and submission requirements, 2026.
2. RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*.
3. Synopsys Design Compiler synthesis, timing, area, constraint, and power reports generated for the final Group 1 implementation.
4. Group 1 cache and branch-predictor sweep records and final presentation.